Fashion Recommendation Project

End to End Implementation Guide

One week build plan with Google Colab GPU access
Prepared 1 October 2026

Target outcome: an interactive prototype that understands clothing requests, remembers explicit preferences and liked products, and returns five relevant catalog items with images and grounded explanations.

Project title: LLM Assisted Personalized Fashion Recommendation Using User Preferences and Semantic Retrieval.

Build the first version entirely inside one Colab notebook. Use notebook widgets for the interface, keep the models and recommendation logic in the same runtime, and export the finished project. This keeps the one-week implementation manageable. A separately hosted website is a later extension.

PREPARE ONCE

Clean catalog → Build item text → Save embeddings

EACH REQUEST

Parse preferences → Filter and rank → Show and save likes

Component	Chosen implementation
Catalog	Fashion Product Images Small; fixed subset of 5,000 to 10,000 clothing and footwear items [1]
Language model	Qwen2.5-3B-Instruct for preference extraction and short explanations [2]
Text embeddings	all-MiniLM-L6-v2 for product and request similarity [3]
Interface and storage	Colab notebook widgets, SQLite profiles, and artifact backups to Drive [4, 5]

What the first version includes

Category, color and usage preferences; free-text requests; multi-turn corrections; saved likes; product images; ranked recommendations; explanations; and a small reproducible evaluation.

Images are displayed, while retrieval uses product text and metadata. The catalog has no existing user interaction history or size, price and stock columns. Do not claim collaborative filtering, fit prediction, current stock checking, or outfit compatibility from this implementation. [1]

This guide specifies work to implement. It does not report a completed application, measured accuracy, or guaranteed performance. The base paper is a research reference; this prototype uses a simpler architecture. [6]

Steps 1 and 2 Set up and collect data

Step 1 Create one reproducible Colab workspace

- Create Fashion_Recommender.ipynb in Google Drive. Select a GPU runtime, then record the assigned GPU, Python version and available memory. GPU availability can vary. [4]
- Create notebook sections in this order: Setup, Catalog, Embeddings, Profiles, LLM, Ranking, Explanations, Interface, Evaluation, Export.
- Use a local runtime folder such as /content/fashion_project. Keep backup archives and the notebook in a dedicated Drive folder. Extract images to the runtime disk for use.

Install the project libraries in an initial notebook cell. Keep the Colab-provided PyTorch installation unless a verified compatibility issue requires a change.

```
%pip install pandas numpy pillow sentence-transformers
%pip install transformers accelerate pydantic ipywidgets
```

These are starting dependencies. After a successful clean run, record the actual versions in requirements-lock.txt and the model revisions in config.json. SQLite is available through Python's standard library.

Completion check: The notebook detects a GPU, imports all packages, and can save and reload a test file. Load Qwen once and generate a short response now, so model or memory problems are discovered on Day 1.

Step 2 Obtain the catalog and inspect its fields

- Download Fashion Product Images Small from its original Kaggle page [1]. Retain the original archive and record the download date. Use the supplied styles.csv and images folder.
- Read the CSV with a CSV parser. Validate the expected ten fields before further processing. If malformed records appear, record their count and resolve or explicitly exclude them; do not silently skip them.

```
id, gender, masterCategory, subCategory, articleType,
baseColour, season, year, usage, productDisplayName
```

- Store id as a stable string. Match each product to images/<id>.jpg and check that the file can be decoded. Preserve raw values and create separate normalized values for matching.
- Inspect actual category, color and usage values. Generate interface choices from these values rather than inventing labels that are absent from the catalog.

Deliverable: a data audit with raw row count, duplicate IDs, malformed rows, missing attributes, invalid images and final usable count. Product attributes can be incomplete; an unknown value must not pass a strict filter that requires known evidence.

Completion check: You can display ten random products with matching IDs, names, images and attributes, and explain every row removed during cleaning.

Steps 3 and 4 Prepare the retrieval catalog

Step 3 Select and freeze a manageable subset

- Keep Apparel and Footwear products. Remove duplicate product IDs and unusable images. Exclude rows without enough metadata to display and categorize the item.
- Select up to 10,000 items using a fixed seed such as 42. Sample across articleType groups so the subset contains several product types; preserve the chosen product IDs.
- Save catalog.csv and catalog_ids.json in the same stable order. Freeze this catalog before evaluation, so all compared systems search exactly the same items.
- Create a short description from recorded fields only. Do not ask the LLM to invent material, quality or detailed occasion labels to fill missing data.

```
Name: Turtle Check Men Navy Blue Shirt
Type: Shirts | Color: Navy Blue | Usage: Casual
Category: Apparel | Season: Fall
```

Suggested files: catalog.csv, catalog_ids.json, data_audit.json and catalog_manifest.json. The manifest should record the source, seed, selection rules, file hashes and actual item count.

Completion check: Every selected ID is unique, has a valid image, and can be joined back to its original record.

Step 4 Compute and save product embeddings

- Load sentence-transformers/all-MiniLM-L6-v2. Encode the product descriptions in batches and normalize the vectors. This model produces 384-dimensional text embeddings. [3]
- Save embeddings.npy as float32. Preserve the exact relationship between embedding row j and catalog row j. Save the embedding model revision and description template.
- For a request, encode its text using the same model and normalization. A dot product between normalized vectors gives cosine similarity.
- Use a simple NumPy matrix multiplication for this catalog size. A vector database is unnecessary for the initial implementation. Compute the product matrix once, then load it when the notebook starts.

```
assert len(catalog) == len(catalog_ids) == len(embeddings)
assert embeddings.shape[1] == 384
assert catalog["id"].tolist() == catalog_ids
```

Keep MiniLM on CPU if that helps leave GPU memory for Qwen. Product vectors may be generated on the GPU during preparation and then reused on the CPU during recommendation.

Completion check: A direct request returns five matching IDs and images after a notebook restart without recomputing the catalog embeddings. Treat similarity as a ranking score, never as an accuracy percentage.

Steps 5 and 6 Represent and interpret preferences

Step 5 Separate current requirements from saved taste

Maintain two distinct states. **Active constraints** describe the current request: item type, included colors, excluded colors and usage. **Saved preferences** describe preferred categories or colors and previously liked product IDs. Current explicit constraints always take priority.

For a new user, rank from their request and onboarding preferences. For a returning user, use likes as a soft ranking signal. Asking for shirts must not be blocked because they previously liked shoes. An anonymous profile ID is enough for the demo.

Step 6 Use Qwen to propose a validated state update

- Load Qwen2.5-3B-Instruct with its tokenizer and official chat template. Use inference mode, a short context and deterministic decoding. Start with batch size one and short outputs; test the actual assigned GPU. [2]
- Provide the current constraints, the allowed catalog values and the latest user message. Request JSON only. Use a schema that supports setting, adding, removing and clearing values.

```
{
  "reset": false,
  "unsupported": [],
  "updates": [
    {"field": "articleType", "op": "set",
     "values": ["Shirts"]},
    {"field": "exclude_colors", "op": "add",
     "values": ["Red"]}
  ]
}
```

An omitted field means keep its current value. Define set as replacement, add as union, remove as subtraction and clear as an empty list. Reset clears the current conversation constraints; clearing the stored profile is a separate user command.

User says	Expected change
Black instead	Replace the included colors with Black.
Avoid red	Add Red to excluded colors.
Any color	Clear both included and excluded color constraints.
Only formal shirts	Set articleType to Shirts and usage to Formal.

Parse with json.loads and validate with Pydantic. Accept only approved fields, operations and catalog values. Use a small synonym map, such as navy to Navy Blue. Reject conflicting rules. Return unsupported requests in the unsupported list and require the user to revise them. Commit only a valid update; on failure, retry once and then expose manual controls.

Completion check: Five test conversations preserve unchanged preferences, handle negation and resets, and leave the previous valid state intact after malformed output. Unsupported requests such as size or price receive a clear unsupported-field response.

Steps 7 and 8 Rank and explain real products

Step 7 Apply constraints before scoring

Build an eligible candidate mask from the validated state. Use OR within a list of accepted values and AND across different fields. Excluded colors always fail. A missing attribute fails any strict constraint that depends on it.

Build the query vector q from the merged current requirements. Preserve compatible context across turns and remove superseded requirements. Do not embed only the latest fragment, such as "black instead". If no request or saved preferences exist, ask the user to choose a category first.

Build the user vector u by averaging normalized vectors of liked products, then normalizing the mean. Use explicit saved preferences when no likes exist. If no usable profile exists, use query similarity alone.

```
score(item) = 0.8 * cosine(q, item_vector)
            + 0.2 * cosine(u, item_vector)
```

These weights are an initial design choice, not an experimentally established optimum. Choose any tuning values on development examples, then freeze them before testing. If the profile mean has zero norm, fall back to request similarity.

- Sort only eligible items by score. Break ties consistently by product ID. Remove duplicate IDs and explicitly dismissed items; return up to five products.
- If fewer than five candidates match, show the available matches. If none match, report no matches and offer explicit filter changes. Never relax the constraints silently.
- Likes influence the next recommendation request. Record them as user feedback; do not treat missing likes as negative ratings or describe them as a large training dataset.

Completion check: No returned item violates an explicit filter. Two profiles can rank the same eligible set differently, while a new user can still receive useful results.

Step 8 Generate short explanations from evidence

Send only selected product IDs, recorded attributes and the relevant preference matches to Qwen. Ask for one brief explanation per ID. Do not send the entire catalog.

```
Allowed example
"This is a navy shirt marked Casual, matching your request."

Unsupported example
"This premium cotton shirt will fit you perfectly."
```

Require explanations keyed by the selected IDs. Reject IDs outside the selected set. Limit statements to an allowlist of known attribute matches and validate mentioned attribute values; use a deterministic metadata template if validation fails. ID validation alone does not establish factual correctness.

Completion check: Each explanation refers to a returned item and recorded attributes. A generation error still leaves the selected products visible with factual template explanations.

Step 9 Build the interface and preserve state

Build the interaction inside the notebook

Use ipywidgets controls and an Output area in Colab [5]. Keep setup and model-loading cells separate from the visible demo controls. This is a notebook-based interactive prototype. A standalone Streamlit website can reuse the functions later.

- **Profile controls:** choose or create an anonymous profile; edit saved preferences; clear that profile only through a separate explicit control.
- **Request controls:** text input, category and color selectors, usage selector, Recommend button, and Clear current request button.
- **Results:** up to five product images, names, attributes, brief explanations and Like or Dismiss buttons. Show the active constraints beside the results.
- **States:** model loading, working, no matches, invalid request, unsupported attribute and recovery through manual filters. Disable repeated submissions while generation is running.

Use small functions with clear responsibilities

Function	Responsibility
prepare_catalog	Validate the source and produce the frozen catalog.
build_embeddings	Encode items and save the ordered vector matrix.
extract_update	Produce a proposed preference update from chat.
validate_and_apply	Validate the update and produce a new active state.
recommend	Filter the catalog and score eligible items.
explain_items	Produce validated explanations or templates.
save_feedback	Persist a like or dismissal for the active profile.

Persist profiles without losing the session

Use SQLite on the runtime disk. Suggested tables are profiles(profile_id, preferences_json, updated_at) and feedback(profile_id, item_id, action, timestamp). Make likes idempotent so repeated clicks do not overweight an item. Keep active conversational state separately per profile.

Back up the database with SQLite's backup API to a snapshot, then copy that snapshot to Drive. Save catalog artifacts and the notebook too. On restart, restore files, verify the catalog and embedding mapping, load models, and initialize the controls. Colab runtime files are temporary. [4]

Completion check: Switching profiles does not mix likes or constraints. After restart, restore the profile and reissue the same request to reproduce ranking. Missing models or GPU failures produce an explicit limited-mode response, not a false claim that the LLM ran.

Step 10 Evaluate each part honestly

Freeze the catalog, model revisions, extraction prompt and ranking settings before testing. Prepare 10 development requests for tuning and a separate 40-request test set. Keep paraphrases of the same scenario in one split. Save requests and expected constraints in JSON.

Prepare four groups of test requests

Use 10 direct requests, 10 exclusions or multi-turn corrections, 10 paraphrases, and 10 edge cases covering resets, unsupported fields, conflicting constraints and no matches. Label the expected state and expected handling manually.

Compare retrieval under the same conditions

Variant	Definition
A Keyword baseline	Token matching over product text after the same gold constraints.
B Semantic retrieval	MiniLM similarity after those same gold constraints.
C Personalized retrieval	Variant B plus saved preferences and real prior likes.
D Full pipeline	LLM-extracted constraints followed by variant C and explanations.

Use gold constraints for A, B and C to isolate ranking. Compare D with C to measure the downstream effect of predicted preferences. For a small trial with 5 to 10 volunteers, collect likes first and freeze profiles. Exclude onboarding likes and previously judged items from every method's evaluation candidates. Never feed test judgements back into ranking. Label this an exploratory study.

Record these measurements

- **Extraction accuracy:** fraction of test requests whose complete resulting constraint state matches the expected state. Separately report valid-JSON rate and correct rejection of unsupported requests.
- **Constraint violations:** violating items divided by returned items on supported requests. The implementation target is zero; report the observed count and denominator.
- **Relevance:** pool and deduplicate top-five results, hide method names, and ask judges whether each item fits the request and user preference. On queries with five results, Precision at 5 is relevant items divided by five. Report fewer-result and no-match cases separately.
- **Explanation errors:** explanations with unsupported factual claims divided by explanations inspected. Include manual review; the LLM must not be its own sole judge.
- **Latency:** measure the complete request time, including extraction and explanation. Report warm median and 95th percentile separately from model-loading and indexing time, with GPU and sample count.

Synthetic profiles may test software behavior, but they do not establish real-user preference quality. Do not claim purchase prediction, recommendation accuracy from cosine scores, or improvement over the paper without matching its data and protocol.

Completion check: Save per-request results, human judgements, failures, timings and an aggregate comparison table. Retain negative results and limitations.

Steps 11 and 12 Package and demonstrate

Step 11 Export a project another person can run

```
fashion_project/
  Fashion_Recommender.ipynb
  README.md
  requirements-lock.txt
  config.json
  artifacts/
    catalog.csv
    catalog_ids.json
    embeddings.npy
    catalog_manifest.json
    data_audit.json
  evaluation/
    development_requests.json
    test_requests.json
    predictions.jsonl
    human_judgements.csv
    metrics.json
  backups/
    profiles_snapshot.sqlite
```

The README should list the original dataset link, permitted download instructions, expected folder layout, model names and revisions, notebook run order, and recovery steps. Follow dataset terms when packaging images; do not assume redistribution is permitted. Keep credentials out of shared notebooks.

Run from a fresh Colab runtime using only the documented steps and saved artifacts. Record the working package versions. Download a project archive, a report, slides and a short demo recording. Verify the archive opens before submission.

Step 12 Follow the seven day schedule

Day	Work	Done when
1	Setup, download and cleaning	Fixed catalog and data audit exist.
2	Embeddings and retrieval	Saved vectors produce reproducible results.
3	Profiles and ranking	Likes affect eligible-item ordering.
4	LLM and validation	Requests and corrections update valid state.
5	Interface and recovery	A complete request-to-feedback demo works.
6	Evaluation	Frozen tests produce a comparison table.
7	Report and submission	Fresh-run check and recording are complete.

Use a short demonstration sequence

Create a profile and like several catalog items. Request casual shirts. Refine the request to black only. Show that the constraints and ranking update. Demonstrate no-match handling, switch to another profile, then show the measured baseline comparison. Use products known to exist in the frozen catalog, and label any pre-recorded output as such.

Completion check: A reviewer can reproduce the notebook, inspect the evaluation files and distinguish implemented behavior from future extensions.

Report outline and final checks

Write the report around what you actually built

Use this order: problem and objectives; related work and base paper; dataset and cleaning; architecture; preference extraction; retrieval and personalization; interface; experimental setup; results; limitations; conclusion and future work. Include the exact catalog size, source fields, model revisions, test-set size and hardware used.

LLM-SCRec generates style representations and integrates them with recommendation backbones. Your project uses an LLM for conversational preference interpretation and grounded explanations, with content-based ranking. State this difference clearly and cite the paper. [6]

Proposed abstract

This project proposes an LLM-assisted personalized fashion recommendation system that combines conversational preference extraction with content-based retrieval. A pretrained language model interprets user requests about clothing categories, colors and usage. A sentence embedding model ranks eligible catalog products using the request, explicit preferences and liked items. The system presents product images and explanations grounded in recorded attributes and supports conversational refinement. An interactive Colab prototype will be evaluated using preference-extraction accuracy, constraint satisfaction, human-judged relevance and response time. The evaluation will compare keyword retrieval, semantic retrieval and personalized retrieval under a fixed catalog.

After implementation, change future-tense statements to completed work only where supported by actual results. Keep collaborative filtering and outfit compatibility as future work unless implemented and evaluated.

Submission checklist

- The title, abstract, architecture and implemented features agree.
- No generated product IDs, unsupported size or price claims, or invented evaluation results appear.
- The notebook starts from a fresh runtime and restores its artifacts and profiles.
- Source links, dataset audit, model settings, comparison results and limitations are included.
- The final archive, report, slides and recorded demonstration are backed up.

Sources and implementation references

[1] Fashion Product Images Small

[2] Qwen2.5 3B Instruct model card

[3] MiniLM L6 v2 model card

[4] Google Colab resource and persistence FAQ

[5] Colab forms and notebook widgets

[6] LLM SCRec base paper

Base paper: Wang, X., Li, Y., Zhou, K., and Tu, W. (2025). LLM-SCRec: LLM-Powered Style-Context Insights for Fashion Recommendation. IEEE Access, 13, 192143-192154. DOI 10.1109/ACCESS.2025.3628684.